

POS SERVICE

Guia Interna de Componentes y Flujos

Version 1.0 - Julio 2026 - Uso personal / aprendizaje

Guia tecnica para aprendizaje personal: arquitectura interna, componentes, flujos de datos y patrones de diseno del repositorio POS SERVICE.

Indice

1. Mapa del repositorio
2. Infraestructura backend
3. ms-auth - Autenticacion JWT
4. pos-service - Nucleo del negocio
5. ms-clientes e integracion Feign
6. Frontend Angular - Arquitectura interna
7. Flujos internos detallados
8. Componentes frontend por pantalla
9. Conceptos clave para aprendizaje
10. Referencias de archivos clave

1. Mapa del repositorio

Estructura raiz

POS SERVICE/

- Config/ - YAML de configuracion centralizada
- ms/ - Backend dockerizado (microservicios)
- pos-frontend/ - Frontend Angular 20
- docs/ - Documentacion y scripts PDF
- README.md - Guia de instalacion Docker

Backend (ms/)

- api-gateway/ - Spring Cloud Gateway, puerto 4040
- registry-server/ - Netflix Eureka, puerto 8761
- config-server/ - Spring Cloud Config
- ms-auth/ - Autenticacion JWT
- pos-service/ - Nucleo POS (ventas, caja, catalogo)
- ms-clientes/ - Microservicio de clientes
- sql/ - Schema y seed BD pos_inventory
- sql-clientes/ - Schema y seed BD clientes_db
- docker-compose.yml - Orquestacion de contenedores
- scripts/ - Utilidades auxiliares

Frontend (pos-frontend/src/app/)

- app.ts / app.config.ts / app.routes.ts - Bootstrap y rutas
- core/auth/ - AuthService, guard, interceptor
- core/layout/ - MainLayoutComponent (sidenav + toolbar)
- core/services/ - Servicios HTTP al backend
- features/ - Pantallas por dominio (pos, orders, etc.)
- shared/components/ - UI reutilizable (dialogs, thumbs)
- shared/models/ - Interfaces TypeScript
- shared/enums/ - OrderType, CashBoxStatus, MovementType

2. Infraestructura backend

API Gateway

Archivo clave: ms/api-gateway/.../beans/GatewayBeans.java

Define tres rutas principales con load balancing via Eureka (lb://):

/auth-server/auth/** -> lb://auth-server

/pos-venta-service/** -> lb://pos-service

/clientes-service/** -> lb://ms-clientes

Perfil Docker: eureka-on. CORS en CorsConfig.java via CORS_ORIGIN.

AuthFilter (Gateway)

Archivo: ms/api-gateway/.../filters/AuthFilter.java

Intercepta peticiones a rutas protegidas. Extrae Bearer token del header Authorization y lo valida llamando POST /auth-server/auth/jwt en ms-auth. Rutas publicas (sin filtro): login, register y el propio endpoint jwt.

Si el token es invalido o ausente, retorna 401 sin llegar al microservicio.

Eureka Registry

Servicio de descubrimiento en puerto 8761. Cada microservicio se registra al iniciar. El Gateway usa nombres de servicio (auth-server, pos-service, ms-clientes) para resolver instancias disponibles.

Config Server

Lee configuracion desde Git (Config/ms-auth.yml, pos-service.yml, etc.). En Docker puede deshabilitarse (SPRING_CLOUD_CONFIG_ENABLED=false) y usar YAML embebidos o variables de entorno del docker-compose.

Bases de datos

pos_inventory (PostgreSQL 5433): tablas del POS y user_info de auth.

clientes_db (PostgreSQL 5434): tabla clientes.

Separacion permite escalar o respaldar clientes independientemente.

3. ms-auth - Autenticacion JWT

Paquete y entidades

Paquete: pe.edu.upeu.ms_auth

UserEntity -> tabla user_info (id, username, password hash BCrypt)

UserRepository: acceso JPA

JwtHelper: generacion y validacion HMAC-SHA, expiracion ~1 hora

AuthServiceImpl: logica de login, register, validateToken

AuthController - Endpoints

POST /auth/login -> { accessToken } si credenciales validas

POST /auth/register -> 201, crea usuario

POST /auth/jwt -> valida token (header accessToken), usado por Gateway

GET /auth/users -> lista usuarios (requiere Bearer)

Flujo interno de login

1. AuthController recibe username + password
2. AuthServiceImpl busca en UserRepository
3. BCrypt verifica hash de contraseña
4. JwtHelper genera token con subject=username
5. Frontend guarda token en sessionStorage
6. authInterceptor adjunta Bearer en cada HttpClient request

Configuracion JWT

Secret en application.jwt.secret (min 32 chars).

Variable de entorno: APPLICATION_JWT_SECRET en ms/.env

Seed: admin/admin123 en ms/sql/data.sql

4. pos-service - Nucleo del negocio

Paquete y capas

Paquete: `pe.edu.upeu.pos_service`

Capas classicas Spring:

- `controllers/` - REST endpoints (`@RestController`)
- `services/` - Logica de negocio (`@Service`)
- `repository/` - JPA repositories
- `entity/` - Entidades JPA
- `dto/` - Objetos de transferencia
- `client/` - `ClienteFeignClient` (`OpenFeign`)

Controladores principales

- `AccountController` -> `/account` (cuentas cajero)
- `CategoryController` -> `/category`
- `ProductController` -> `/product` (+ search, by category)
- `ProductImageController` -> `/product-image` (multipart)
- `CategoryImageController` -> `/category-image` (multipart)
- `OrderController` -> `/order` (+ reportes, by cash-box)
- `OrderDetailController` -> `/order-detail`
- `GeneralCashBoxController` -> `/cash-box` (open, close, get open)
- `CashMovementController` -> `/cash-movement`

OrderServiceImpl - Ventas

Archivo: `services/OrderServiceImpl.java`

Responsabilidades:

- Verificar que exista caja OPEN antes de crear orden
- Obtener numero secuencial via `findMaxOrderNumberByCashBox`
- Llamar `ClienteFeignClient` para datos del cliente
- Si Feign falla, usar cliente `id=1` (Clientes Varios)
- Copiar `customer_name` y `customer_document` a la orden (snapshot)
- Por cada detail: leer precio del Product, calcular subtotal
- Sumar total de la orden y persistir Order + OrderDetails

GeneralCashBoxServiceImpl - Caja

- `open()`: valida que no haya otra caja OPEN, crea `GeneralCashBox`
- `close()`: calcula `saldo_final = inicial + ventas + ingresos - egresos`
- `getOpen()`: retorna la caja activa o null

CashMovementServiceImpl

- create(): solo si caja asociada esta OPEN
- Tipos: MovementType.INCOME o MovementType.EXPENSE

Modelo de datos (pos_inventory)

account - Cajeros (email unico)
category - Categorías de productos
imagen_cat - Imágenes de categoría (BYTEA)
product - Productos (precio, category_id, imagen)
product_image - Catalogo imágenes producto (BYTEA)
general_cash_box - Sesiones caja (OPEN/CLOSED, saldos)
pos_order - Ventas (tipo, cliente snapshot, total, numero)
pos_order_detail - Líneas de venta (product, qty, precio, subtotal)
cash_movement - Ingresos/egresos manuales
user_info - Usuarios JWT (ms-auth)

Enums de dominio

CashBoxStatus: OPEN, CLOSED

MovementType: INCOME, EXPENSE

OrderType: SALE, CREDIT, COURTESY, DAMAGED

5. ms-clientes e integracion Feign

ms-clientes

Paquete: `pe.edu.upeu.ms_clientes`

BD propia: `clientes_db` (PostgreSQL 5434)

Entidad Cliente: `nombre_completo`, `tipo_documento`, `numero_documento`, `direccion`, `correo`

ClienteController en `/api/clientes`

Endpoints de clientes

GET `/api/clientes` - Listar todos

GET `/api/clientes/{id}` - Por ID

POST `/api/clientes` - Crear

GET `/api/clientes/documento/{numero}` - Por DNI/RUC

GET `/api/clientes/buscar?nombre=` - Búsqueda parcial

ClienteFeignClient

Archivo: `pos-service/client/ClienteFeignClient.java`

Interface `@FeignClient` apuntando a `ms-clientes` via Eureka.

`OrderServiceImpl` lo usa en `create()` para obtener nombre y documento.

Patron de resiliencia: `try/catch` o `fallback` a cliente `id=1` si el servicio de clientes no responde. Esto evita bloquear ventas por fallo de red.

Por que microservicio separado

Separacion de dominios: `clientes` puede reutilizarse por otros sistemas.

BD independiente: migraciones y backups separados.

Escalabilidad: `ms-clientes` puede escalar sin afectar `pos-service`.

Trade-off: latencia extra en cada venta (llamada HTTP sincronica).

6. Frontend Angular - Arquitectura interna

Bootstrap y configuracion

main.ts -> bootstrapApplication(App, appConfig)

app.config.ts provee:

provideRouter(routes)

provideHttpClient(withInterceptors([authInterceptor]))

provideAnimationsAsync()

App component: solo <router-outlet /> (shell minimo)

Rutas y lazy loading

app.routes.ts define rutas publicas (/login, /register) y layout protegido.

MainLayoutComponent envuelve rutas hijas con authGuard.

Cada feature usa loadComponent para cargar bajo demanda:

/dashboard, /pos, /orders, /cash-box, /cash-movements,

/products, /categories, /clients, /accounts, /reports

No hay NgModules: todo es standalone: true.

Environment

environment.ts:

apiUrl: 'http://localhost:4040'

posPrefix: '/pos-venta-service'

clientesPrefix: '/clientes-service'

authPrefix: '/auth-server/auth'

Todos los servicios construyen URLs como apiUrl + prefix + endpoint.

Capa core/auth

AuthService: login, register, logout, getToken, isAuthenticated

Token y username en sessionStorage

auth.guard.ts: CanActivateFn, redirige a /login si !isAuthenticated

auth.interceptor.ts: HttpInterceptorFn

- Adjunta Bearer excepto en login/register

- En 401: logout() + navigate('/login')

Capa core/services

Un servicio por recurso REST, inyectable con inject():

ProductService, CategoryService, OrderService, OrderDetailService

CashBoxService, CashMovementService, AccountService

ProductImageService, CategoryImageService, ClienteService

Patrón: métodos retornan Observable<T>, componentes suscriben o usan async pipe.

CashBoxStateService - Estado compartido

Archivo: core/services/cash-box-state.service.ts

Usa BehaviorSubject<GeneralCashBox | null>.

Metodos: setOpenCashBox, clearOpenCashBox, openCashBox\$, hasOpenCashBox.

Consumidores:

MainLayoutComponent - banner si no hay caja

PosComponent - bloquea venta

OrdersComponent - filtra por caja abierta

CashMovementsComponent - requiere caja activa

CashBoxComponent - actualiza al abrir/cerrar

Signals vs RxJS

Componentes modernos usan signal() para estado local (carrito, listas, loading).

RxJS se usa en servicios HTTP (Observable) y CashBoxStateService (Subject).

Patron hibrido: subscribe en ngOnInit -> actualiza signals -> template reactivo.

7. Flujos internos detallados

Flujo completo de una venta

1. Usuario abre caja en CashBoxComponent
 - > CashBoxService.open({ accountId, saldoInicial })
 - > CashBoxStateService.setOpenCashBox(response)
2. Usuario navega a PosComponent
 - > CategoryService.getAll(), ProductService.getAll()
 - > ClienteService.getAll() para selector
3. Usuario agrega productos al carrito (estado local en signals)
4. Usuario confirma venta
 - > Valida hasOpenCashBox() y carrito no vacío
 - > OrderService.create({ cashBoxId, clientId, orderType, details[] })
5. authInterceptor agrega Bearer token
6. Gateway AuthFilter valida JWT con ms-auth
7. OrderController -> OrderServiceImpl.create()
 - > Valida caja OPEN en BD
 - > Feign a ms-clientes por clientId
 - > Calcula precios desde Product repository
 - > Persiste Order + OrderDetails
8. Respuesta con id, orderNumber, total
9. PosComponent vacía carrito, muestra Snackbar de éxito

Flujo de apertura y cierre de caja

Apertura:

- CashBoxComponent -> AccountService.getAll() (cuentas cajero)
- > Usuario elige cuenta + saldo inicial
 - > CashBoxService.open() -> GeneralCashBoxServiceImpl
 - > Valida no hay otra OPEN -> INSERT general_cash_box
 - > CashBoxStateService.setOpenCashBox()

Cierre:

- CashBoxService.close(id) -> GeneralCashBoxServiceImpl
- > Suma ventas de pos_order para esa caja
 - > Suma movimientos INCOME/EXPENSE
 - > Calcula saldo_final -> UPDATE status=CLOSED
 - > CashBoxStateService.clearOpenCashBox()

Flujo de imágenes

Upload (administración):

- ImagePickerComponent -> ProductImageService.upload(file)
- > POST multipart /product-image
 - > ProductImageServiceImpl guarda bytes en BYTEA

Display (thumbs):

ProductImageThumbComponent recibe imageId

-> ProductImageService.getBlobUrl(id)

-> GET /product-image/{id} con Bearer (via interceptor)

-> responseType: 'blob'

-> URL.createObjectURL(blob) cacheado en Map<id, url>

Motivo del cache: no envia headers Authorization,
por eso se usa blob URL en lugar de URL directa al API.

Flujo de autenticacion (secuencia)

LoginComponent -> AuthService.login(credentials)

-> POST /auth-server/auth/login (sin Bearer)

-> sessionStorage.setItem('token', accessToken)

-> router.navigate(['/dashboard'])

Peticion protegida:

HttpClient -> authInterceptor -> agrega Bearer

-> Gateway AuthFilter -> POST /auth/jwt valida

-> forward a microservicio -> respuesta

Token expirado:

Backend/Gateway 401 -> interceptor logout -> /login

8. Componentes frontend por pantalla

Tabla de pantallas

/login	LoginComponent	AuthService
/register	RegisterComponent	AuthService
/dashboard	DashboardComponent	CashBoxService, OrderService
/pos	PosComponent	Category, Product, Order, Cliente, CashBoxState + CategoryImageThumb, ProductImageThumb
/orders	OrdersComponent	OrderService, OrderDetailService + OrderDetailDialog, ConfirmDialog
/cash-box	CashBoxComponent	CashBoxService, AccountService, CashBoxState
/cash-movements	CashMovementsComponent	CashMovementService, CashBoxState
/products	ProductsComponent	ProductService, ProductImageService + ProductFormDialog, ImagePicker, ConfirmDialog
/categories	CategoriesComponent	CategoryService, CategoryImageService + CategoryFormDialog, CategoryImagePicker
/clients	ClientsComponent	ClienteService + ClientFormDialog
/accounts	AccountsComponent	AuthService.getRegisteredUsers()
/reports	ReportsComponent	OrderService (report list + total)

Componentes compartidos (shared/)

PageHeaderComponent - titulo, subtítulo, slot de acciones
 ConfirmDialogComponent - confirmación genérica (MatDialog)
 ProductImageThumbComponent - miniatura blob de producto
 CategoryImageThumbComponent - miniatura blob de categoría
 ProductFormDialogComponent - formulario crear/editar producto
 CategoryFormDialogComponent - formulario crear/editar categoría
 ClientFormDialogComponent - formulario crear cliente
 OrderDetailDialogComponent - detalle de líneas de orden
 ImagePickerComponent / CategoryImagePickerComponent - galería y upload

Distinción user_info vs account

IMPORTANTE para no confundir conceptos:

user_info (ms-auth): usuarios que hacen login JWT. Pantalla /accounts los lista.

account (pos-service): operadores de caja vinculados a general_cash_box.

Se gestionan vía AccountService, usados solo al abrir caja.

Un mismo operador humano podría tener login JWT y cuenta POS separadas.

MainLayoutComponent

core/layout/main-layout.component.ts

MatSidenav con links a todas las rutas.

Toolbar con username (AuthService) y boton logout.

Banner de advertencia si CashBoxStateService indica sin caja abierta.

Router-outlet para contenido de features.

9. Conceptos clave para aprendizaje

Snapshot de cliente vs referencia por ID

pos_order guarda customer_name y customer_document copiados al vender.

Ventaja: el historial de ventas no cambia si el cliente actualiza sus datos.

Desventaja: datos duplicados; no hay FK en BD hacia clientes_db (otro servicio).

Por que el interceptor excluye login/register

Esas rutas son publicas en el Gateway. Enviar un token viejo o invalido podria causar rechazos innecesarios. El interceptor detecta la URL y no agrega Authorization header.

Gateway como unico punto de entrada

El frontend nunca llama directamente a puertos 8082/8083/3030.

Beneficios: CORS centralizado, validacion JWT unica, ocultamiento de topologia interna, posibilidad de rate limiting futuro.

OpenFeign vs HttpClient frontend

Feign (backend): comunicacion sincronica entre microservicios Java.

HttpClient (frontend): comunicacion con Gateway desde Angular.

Ambos son clientes HTTP pero en capas diferentes de la arquitectura.

Standalone components en Angular 20

Sin AppModule: cada componente declara sus imports (Material, Common, etc.).

loadComponent en rutas reemplaza loadChildren de modulos.

inject() reemplaza constructor injection en muchos casos.

signals() para estado reactivo sin NgRx para este tamaño de app.

Docker Compose y seeds

Al levantar contenedores, scripts SQL en sql/ y sql-clientes/ inicializan schema y datos de prueba (admin, categorias, productos, cliente varios).

Healthchecks en docker-compose esperan que PostgreSQL este listo antes de iniciar microservicios.

10. Referencias de archivos clave

Backend

ms/docker-compose.yml
ms/sql/create_schema.sql
ms/sql/data.sql
ms/api-gateway/.../beans/GatewayBeans.java
ms/api-gateway/.../filters/AuthFilter.java
ms/ms-auth/.../controllers/AuthController.java
ms/ms-auth/.../helpers/JwtHelper.java
ms/pos-service/.../services/OrderServiceImpl.java
ms/pos-service/.../services/GeneralCashBoxServiceImpl.java
ms/pos-service/.../client/ClienteFeignClient.java
ms/ms-clientes/.../controller/ClienteController.java

Frontend

pos-frontend/src/app/app.routes.ts
pos-frontend/src/app/app.config.ts
pos-frontend/src/environments/environment.ts
pos-frontend/src/app/core/auth/auth.service.ts
pos-frontend/src/app/core/auth/auth.guard.ts
pos-frontend/src/app/core/auth/auth.interceptor.ts
pos-frontend/src/app/core/services/cash-box-state.service.ts
pos-frontend/src/app/core/layout/main-layout.component.ts
pos-frontend/src/app/features/pos/pos.component.ts
pos-frontend/src/app/features/cash-box/cash-box.component.ts

Como profundizar

1. Levantar stack con docker compose up en ms/
2. Abrir Swagger en pos-service (si expuesto) o usar Postman contra :4040
3. Ejecutar frontend con npm start en pos-frontend/
4. Poner breakpoints en OrderServiceImpl y PosComponent
5. Trazar una venta completa con DevTools Network tab
6. Revisar logs de Gateway para ver validacion JWT en accion